the 'kvm' group:

```
$ sudo adduser user kvm
```

Logout and login again, and you can then start the VMs without root privileges.

```
$ qemu-system-x86_64 -boot d -cdrom /images/debian-lenny.iso \
-hda debian-lenny.img
```

On Fedora, this is:

```
# qemu-kvm -boot d -cdrom /images/debian-lenny.iso \
-hda debian-lenny.img
```

This command starts a VM session. Once the install is completed, you can run the guest with the following command:

```
$ qemu-system-x86_64 debian-lenny.img
```

You can also pass the *-m* parameter to set the amount of RAM the VM gets. The default value is 128 M.

Recent KVM releases have support for swapping guest memory, so the RAM allocated to the guest isn't pinned down on the host.

Troubleshooting

There will be times when you run into some bugs in VMs with KVM. In such cases, there will be some output in the host kernel logs. That can help you search for similar problems reported earlier and any solutions that might be available. In most cases, upgrading to the latest KVM release and running it might fix the problem.

In case you don't find a solution, running the VM by passing the *-no-kvm* command line to qemu will start it without KVM support. If this also doesn't solve the problem, it means the problem lies in qemu itself.

Another thing to try out might be to pass the *-no-kvm-irqchip* parameter while starting a VM.

You can also ask on the friendly *#kvm* IRC channel at Freenode, or on the *kvm@vger.kernel.org* mailing list, and someone will help you out.

qemu monitor

The qemu monitor can be entered by typing the *Ctrl+Alt+2* key combination when the qemu window is selected, or by passing *-monitor stdio* to the qemu command line. The monitor gives access to some debugging commands and those that can help inspect the state of the VM.

For example, info registers show the contents of the registers of the virtual CPU. You can also attach USB devices to a VM, change the CD image and do other interesting things via the qemu monitor.

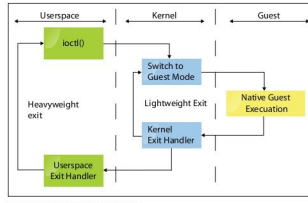

Figure 2: The KVM execution model

Migration of VMs

Migrating VMs is a very important feature for load-balancing, hardware upgrades, and software upgrades for zero or very minimal downtime. The guests are migrated from one physical machine to another, and the original machine can then be taken down for maintenance.

The advantage in the KVM approach is that the guests are not involved at all in the migration. Also, nothing special needs to be done to tunnel a migration through an SSH session, to compress the image being migrated, etc. You can even pass the image through any program you want before it's transmitted to the target machine. The UNIX philosophy holds good here as well. Also, unless specific hardware or host-specific features are enabled, the migration can be made between any two machines. Moreover, stopped guests can be migrated as well as live guests. We add the migration facility within qemu, so no kernel-side changes are needed to enable it. The device state-syncs to achieve migration, and the VM state is seamlessly provided and managed within userspace.

On the target machine, run qemu with the same command-line options as was given to the VM on the source machine, with additional parameters for migration-specific commands:

```
$ qemu-system-x86_64 -incoming <protocol://params>
```

Example:

```
$ qemu-system-x86_64 -m 512 -hda /images/f10.img -incoming params
```

On the source machine, start migration using the 'migrate' qemu monitor command:

```
(qemu) migrate <migration-protocol://params>
```

An example of the source qemu monitor command:

```
(qemu) migrate tcp://dst-ip:dst-port
```